In [ ]:
```
import pandas as pd
dataset=pd.read_csv("insurance.csv")
dataset
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

In [ ]:
```
x=dataset[['age', 'bmi', 'children','sex_male', 'smoker_yes','region_northwest','region_southeast','region_southwest']]
y=dataset['charges']
print(x)
print(y)
```

In [4]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_state=42)
```

In [5]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

In [37]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import AdaBoostRegressor
param_grid={
    'random_state':[10,100,1000],
    'n_estimators':[100,1000,2000,3000]
}
grid=GridSearchCV(AdaBoostRegressor(),param_grid,refit=True,verbose=3,n_jobs=-1)
grid.fit(x_train,y_train)
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
```

Out[37]:

▸ GridSearchCV

▸ best_estimator_: AdaBoostRegressor

▸ AdaBoostRegressor

In [ ]:
```
grid_pred=grid.predict(x_test)
grid_pred
```

In [39]:
```
from sklearn.metrics import r2_score
result=r2_score(grid_pred,y_test)
result
```

Out[39]:

```
0.8076450949778454
```

In [42]:
```
re=grid.cv_results_
print("The R_scorevalueforbestparameter{}:".format(grid.best_params_),result)
```

```
The R_scorevalueforbestparameter{'n_estimators': 100, 'random_state': 1000}: 0.8076450949778454
```

In [43]:
```
table=pd.DataFrame.from_dict(re)
table
```

Out[43]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_n_estimators	param_random_state	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score	split4_test_score	mean_test_score	std_test_score	rank_test_s
0	0.060078	0.014098	0.003645	0.001131	100	10	{'n_estimators': 100, 'random_state': 10}	0.766775	0.883444	0.766022	0.760344	0.758225	0.786962	0.048351	
1	0.028875	0.005571	0.003212	0.000874	100	100	{'n_estimators': 100, 'random_state': 100}	0.796673	0.845092	0.763092	0.788571	0.828299	0.804346	0.029133	
2	0.026222	0.006171	0.002986	0.000644	100	1000	{'n_estimators': 100, 'random_state': 1000}	0.809593	0.880146	0.748619	0.788531	0.817284	0.808835	0.042885	
3	0.028231	0.010648	0.002728	0.000677	1000	10	{'n_estimators': 1000, 'random_state': 10}	0.766775	0.883444	0.766022	0.760344	0.758225	0.786962	0.048351	
4	0.027688	0.004609	0.003019	0.000700	1000	100	{'n_estimators': 1000, 'random_state': 100}	0.796673	0.845092	0.763092	0.788571	0.828299	0.804346	0.029133	
5	0.023777	0.006372	0.002618	0.000610	1000	1000	{'n_estimators': 1000, 'random_state': 1000}	0.809593	0.880146	0.748619	0.788531	0.817284	0.808835	0.042885	
6	0.027404	0.010180	0.002717	0.000634	2000	10	{'n_estimators': 2000, 'random_state': 10}	0.766775	0.883444	0.766022	0.760344	0.758225	0.786962	0.048351	
7	0.026022	0.006804	0.002619	0.000370	2000	100	{'n_estimators': 2000, 'random_state': 100}	0.796673	0.845092	0.763092	0.788571	0.828299	0.804346	0.029133	
8	0.023420	0.006110	0.002492	0.000416	2000	1000	{'n_estimators': 2000, 'random_state': 1000}	0.809593	0.880146	0.748619	0.788531	0.817284	0.808835	0.042885	
9	0.026698	0.009451	0.002834	0.000806	3000	10	{'n_estimators': 3000, 'random_state': 10}	0.766775	0.883444	0.766022	0.760344	0.758225	0.786962	0.048351	
10	0.026680	0.006172	0.002958	0.000709	3000	100	{'n_estimators': 3000, 'random_state': 100}	0.796673	0.845092	0.763092	0.788571	0.828299	0.804346	0.029133	
11	0.022191	0.005510	0.002060	0.000620	3000	1000	{'n_estimators': 3000, 'random_state': 1000}	0.809593	0.880146	0.748619	0.788531	0.817284	0.808835	0.042885	

In [ ]:

In [ ]:
```
import pandas as pd
dataset=pd.read_csv("insurance.csv")
dataset
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

In [ ]:
```
x=dataset[['age', 'bmi', 'children','sex_male', 'smoker_yes','region_northwest','region_southeast','region_southwest']]
y=dataset['charges']
print(x)
print(y)
```

In [3]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_state=42)
```

In [4]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

In [12]:
```
from sklearn.ensemble import GradientBoostingRegressor
reg=GradientBoostingRegressor(n_estimators=500,
    max_depth=4,
    min_samples_split=5,
    learning_rate=0.01,
    loss="squared_error")
reg.fit(x_train,y_train)
```

Out[12]:

▾ GradientBoostingRegressor

▸Parameters

▸Fitted attributes

In [13]:
```
y_pred=reg.predict(x_test)
y_pred
```

```
from sklearn.metrics import r2_score
result=r2_score(y_pred,y_test)
result
```

Out[13]:

```
0.8589121129980305
```

In [ ]:

In [6]:
```
import lightgbm as lgb
print(lgb.__version__)
```

```
4.7.0
```

In [ ]:
```
import pandas as pd
dataset=pd.read_csv("insurance.csv")
dataset
```

In [ ]:
```
dataset=pd.get_dummies(dataset,drop_first=True)
dataset
```

In [ ]:
```
x=dataset[['age','bmi','children','sex_male','smoker_yes','region_northwest','region_southeast','region_southwest']]
y=dataset[['charges']]
print(x)
print(y)
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_state=42)
```

In [8]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

In [12]:
```
from lightgbm import LGBMRegressor
reg=LGBMRegressor(
    objective='regression',
    metric='rmse',
    boosting_type='gbdt',
    num_leaves=31,
    learning_rate=0.05,
    feature_fraction=0.9
)
reg.fit(x_train,y_train)
```

```
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.9
[LightGBM] [Warning] feature_fraction is set=0.9, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.9
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000205 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 325
[LightGBM] [Info] Number of data points in the train set: 1070, number of used features: 8
[LightGBM] [Info] Start training from score 13346.089733
```

```
C:\Users\User\anaconda3\Lib\site-packages\sklearn\utils\validation.py:1365: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

Out[12]:

▾ LGBMRegressor

▸Parameters

▸Fitted attributes

In [ ]:
```
y_pred=reg.predict(x_test)
y_pred
```

In [14]:
```
from sklearn.metrics import r2_score
result=r2_score(y_pred,y_test)
result
```

Out[14]:

```
0.8497833819826975
```